

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Integración del proyecto con Base de Datos

Programación II

Saimon Daniel Gonzalez López

9989-25-10617



Captura de la base de datos con los registros almacenados

detalle_factura x

Limit to 1000 rows

```
1 • SELECT * FROM sistema_ventas.detalle_factura;
```

Result Grid

	id	factura_id	producto	cantidad	precio_unitario	subtotal
▶	2	2	MENSUALIDAD	1	1020.00	1020.00
	3	3	MENSUALIDAD	50	1020.00	51000.00
*	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid

Form Editor

detalle_factura factura x

Limit to 1000 rows

```
1 • SELECT * FROM sistema_ventas.factura;
```

Result Grid

	id	fecha	total
▶	2	2026-08-28 06:00:00	1020.00
	3	2026-08-28 06:00:00	51000.00
*	NULL	NULL	NULL

¿Cómo realizamos la conexión a la base de datos?

Para conectar la aplicación Java con MySQL utilizamos la API de JDBC a través de una clase dedicada llamada `DatabaseConnection`. El proceso funciona de la siguiente manera:

- **Carga del Driver:** Cargamos en memoria el controlador oficial de MySQL (`mysql-connector-j` versión 8.3.0) mediante `Class.forName()`. Esto permite que Java entienda el protocolo de comunicación de MySQL.
- **Configuración de la URL:** Definimos una dirección JDBC (`jdbc:mysql://localhost:3306/sistema_ventas`) agregando ajustes para evitar problemas comunes de compatibilidad, como la zona horaria (`serverTimezone=UTC`), el uso de SSL (`useSSL=false`) y la autenticación.
- **Entrega de conexiones:** Cada vez que la aplicación necesita guardar, leer o borrar algo, llama al método estático `DatabaseConnection.getConnection()`. Este se encarga de abrir un canal seguro de comunicación con el puerto de MySQL utilizando el usuario y la contraseña configurados.

Componentes y clases agregados para la persistencia

Para organizar el código de forma limpia y mantener el patrón MVC (Modelo-Vista-Controlador), implementamos el patrón DAO (Data Access Object). Las clases clave encargadas de la base de datos son:

- **DatabaseConnection:** Es la puerta de entrada a MySQL. Se encarga únicamente de crear y entregar conexiones JDBC cuando el sistema lo requiere.
- **Invoice e InvoiceDetail:** Son las entidades del modelo. Representan los datos en memoria: `Invoice` maneja la información general de la factura (fecha, total y lista de ítems) e `InvoiceDetail` representa cada renglón de producto (nombre, cantidad, precio y subtotal).
- **InvoiceDao (Interfaz):** Define el contrato con todas las operaciones que el sistema puede hacer en la base de datos (guardar factura, consultar registros al iniciar, eliminar un producto guardado o consultar el correlativo).
- **InvoiceDaoImpl (Implementación):** Contiene las consultas SQL reales (`INSERT`, `SELECT`, `DELETE`) ejecutadas mediante `PreparedStatement` para prevenir inyecciones SQL.

Maneja transacciones de forma manual (`setAutoCommit(false)`): si al guardar una factura falla la inserción de un producto, se ejecuta un `rollback()` para evitar guardar datos incompletos.

Procesa inserciones por lote (`addBatch()`) para mejorar el rendimiento al guardar varios productos a la vez.

Permite eliminar permanentemente registros individuales de MySQL y recargar la lista de productos guardados al abrir la aplicación.